



NAS-SE: Designing A Highly-Efficient In-Situ Neural Architecture Search Engine for Large-Scale Deployment

Qiyu Wan*
NVIDIA and University of Houston
Santa Clara, CA, USA
qiyuw@nvidia.com

Lening Wang
University of Houston
Houston, Texas, USA
lwang56@uh.edu

Jing Wang
Renmin University of China
Beijing, China
jwang@ruc.edu.cn

Shuaiwen Leon Song
Microsoft and University of Sydney
Redmond, USA
leonangel991@gmail.com

Xin Fu
University of Houston
Houston, Texas, USA
xfu8@central.uh.edu

ABSTRACT

The emergence of Neural Architecture Search (NAS) enables an automated neural network development process that potentially replaces manually-enabled machine learning expertise. A state-of-the-art NAS method, namely *One-Shot NAS*, has been proposed to drastically reduce the lengthy search time for a wide spectrum of conventional NAS methods. Nevertheless, the search cost is still prohibitively expensive for practical large-scale deployment with real-world applications. In this paper, we reveal that the fundamental cause for inefficient deployment of One-Shot NAS in both single-device and large-scale scenarios originates from the massive redundant off-chip weight access during the numerous DNN inference in sequential searching. Inspired by its algorithmic characteristics, we depart from the traditional CMOS-based architecture designs and propose a promising processing-in-memory design alternative to perform in-situ architecture search, which helps fundamentally address the redundancy issue. Moreover, we further discovered two major performance challenges of directly porting the searching process onto the existing PIM-based accelerators: severe pipeline contention and resource under-utilization. By leveraging these insights, we propose the first highly-efficient in-situ One-Shot NAS search engine design, named *NAS-SE*, for both single-device and large-scale deployment scenarios. *NAS-SE* is equipped with a two-phased network diversification strategy for eliminating resource contention, and a novel hardware mapping scheme for boosting the resource utilization by an order of magnitude. Our extensive evaluation demonstrates that *NAS-SE* significantly outperforms the state-of-the-art digital-based customized NAS accelerator (*NASA*) with an average speedup of 8.8× and energy-efficiency improvement of 2.05×.

*This work was done when the author was at University of Houston.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0329-4/23/10...\$15.00
<https://doi.org/10.1145/3613424.3614265>

CCS CONCEPTS

• **Hardware** → **Emerging technologies**; • **Computing methodologies** → **Machine learning**.

KEYWORDS

Neural architecture search, Evolutionary algorithm, In-memory computing, Hardware accelerator

ACM Reference Format:

Qiyu Wan, Lening Wang, Jing Wang, Shuaiwen Leon Song, and Xin Fu. 2023. NAS-SE: Designing A Highly-Efficient In-Situ Neural Architecture Search Engine for Large-Scale Deployment. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*, October 28–November 01, 2023, Toronto, ON, Canada. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3613424.3614265>

1 INTRODUCTION

By simplifying the problems as automatic *feature engineering*, Deep Neural Networks (DNNs) have achieved tremendous success in various computer vision tasks such as image classification [17, 35, 38], object detection [15, 20] and visual generation [46, 47]. However, the actual architecture design of a DNN model, known as *architecture engineering* [16, 58], still demands significant amount of efforts and cycles from machine learning experts. Fortunately, the recent emergence of Neural Architecture Search (NAS) [5, 24, 37, 40, 50, 61, 62] has brought the architecture engineering into an era of automation. The goal of NAS is to automatically search for the network architectures that can achieve better performance for the target tasks. By now, the networks designed by the existing NAS technologies have shown a remarkable performance improvement over manually-design networks in numerous application domains [11, 16, 27, 30, 43, 52, 56, 57].

While the DNN design process is fully automated by NAS, the time taken by its entire searching process can be extremely lengthy (e.g., weeks or even months). Conventional NAS methods [62] require exploring numerous neural network architectures from a massive search space and train all of them from scratch to evaluate their performance, which makes the searching procedure prohibitively expensive. To drastically improve search efficiency, *One-Shot NAS* approaches [2–4, 11, 13, 16, 56] have recently gained great popularity and extensively studied at algorithmic level. For instance, a recent One-Shot NAS proposal [16] reduces the search time to

≤ 1 GPU day while achieving higher accuracy (up to 1.6%) compared to the other state-of-the-art NAS-searched models. Superior to the conventional methods, One-Shot NAS only requires to train a *Supernet* once. After this supernet is trained, the architecture searching stage initiates, during which the candidate architectures (defined as *subnets*) can directly inherit weights from the supernet and apply these weights to evaluate their performance. In the meantime, an evolutionary algorithm is applied to iteratively search for the candidate architecture with better evaluation accuracy.

Although One-Shot NAS reduces the overall search cost by converting the numerous network training processes into one-time effort, the remaining search cost is still very high and becomes the major bottleneck hindering its real-world deployment across a large spectrum of devices. Specifically, given the same machine learning task, the optimal architectures corresponding to different hardware devices may turn out to be very different due to the diverse hardware constraints such as inference latency, the required FLOPS achieved by the searched network architecture, etc. Therefore, searching the optimal architectures for various hardware devices further increases the overall searching cost. Based on our quantitative analysis, the state-of-the-art One-Shot NAS seriously suffers from massive cost due to the numerous *DNN inference* throughout the architecture searching stage. For example, FairNAS [11] takes approximately 59 GPU days to capture the optimal architectures corresponding to 50 hardware devices, which involves evaluating 50k subnets during searching.

To clearly identify the root causes for this inefficiency, we first conduct a quantitative analysis on large-scale deployment scenarios through One-Shot NAS searching on digital CMOS-based architecture designs, e.g., general-purpose GPUs and customized accelerators. We observed that the root cause for this inefficiency originates from the massive redundant off-chip weight access caused by a *great degree of similarity* between the architectures of the subnets during the sequential searching process, due to the fact that candidate subnets' weights in One-Shot NAS are directly inherited from the trained supernet. This situation is exacerbated when searching for a large number of deployment scenarios. The recent digital CMOS-based architecture design solution, i.e., NASA [29], reorganized the searching process to reduce the redundancy by exploiting computation sharing and data reuse. However, it has several significant limitations. First, NASA only reduces weight access via fusing in the first few blocks of the candidate subnets, whereas its local weight reuse is at the expense of input/output reuse. Furthermore, the main benefits of their design does not directly come from massive redundant memory-access reduction, which remains as the fundamental bottleneck for searching. Finally, NASA only attempts to optimize searching within one generation for a single search, but neglects the redundancy across a number of searches in a large-scale deployment scenario.

Inspired by the algorithmic characteristics of One-Shot NAS, we propose an alternative design solution that applies the Resistive Random Access Memory (*ReRAM*) crossbars to "One-Shot" store the supernet's weights, and then performs in-situ searching process on ReRAM crossbars. Its feasibility is proved by the fact that the unique feature of "one-time training" in One-Shot NAS makes the writing operations of ReRAM cells an one-time effort. More importantly, ReRAM has the capability of performing in-memory

Matrix Vector Multiplication (MVMs). The major computations, i.e., MVMs, in network evaluation processes can thus be directly conducted on ReRAM crossbar in the *analog domain* by storing the weights as memory cell conductance and serving input data as voltage signals. This feature makes ReRAM outperforms other emerging non-volatile memory technologies and PIM solutions. Recently, many ReRAM-based DNN accelerators have been proposed and have emerged as the main stream solution for in-memory DNN acceleration [1, 9, 21, 34, 36, 49, 54].

Although this basic idea seems promising, we have identified two major challenges that hinder the practical design and deployment of One-Shot NAS on the current ReRAM-based DNN accelerators. First, when concurrently evaluating a group of subnets on ReRAM, the same weights stored on the ReRAM can easily be competed by several subnets at the same time due to the high degree of architectural similarity among the subnets, resulting in abundant pipeline bubbles. This leads to severe resource contention which significantly increases the searching latency. Second, as a major component of One-Shot NAS network architecture, the depthwise convolutional layer can not be effectively mapped onto ReRAM crossbars via the conventional mapping scheme due to its poor computation sharing characteristics, which causes low resource utilization of ReRAM crossbars.

Based on these insights, we propose *NAS-SE*, an efficient in-situ One-Shot NAS Search Engine for single-device and large-scale deployment scenarios. *NAS-SE* intelligently addresses the two challenges above through our sophisticated two-phased network diversification approach and a novel ReRAM mapping technique for depthwise convolution. Specifically, our *two-phased* network diversification technique effectively eliminates the resource competition issue and mitigates the accuracy impact via early-stage across-network similarity reduction (i.e., intra-search grouping), while expanding *NAS-SE*'s capability to handle large-scale searching via late-stage inter-search grouping. Furthermore, by exploiting the kernel-level input data sharing opportunities, our novel hardware mapping scheme boosts the resource utilization in depthwise convolutional layers by an order of magnitude. Finally, *NAS-SE*'s sophisticated software-hardware co-design drastically reduces the overall design complexity and development cost via only minor modifications to ReRAM crossbar's peripherals.

Our extensive evaluation results show that *NAS-SE* achieves significant *speedup* and *energy efficiency* in searching over its general-purpose and customized accelerator counterparts, e.g., on average, 431 $\times$ and 446 $\times$ over the baseline GPU implementation and 8.8 $\times$ and 2.05 $\times$ over NASA, respectively. In summary, this work makes the following contributions:

- Through quantitative analysis, we identified the root cause for inefficient deployment of One-Shot NAS on digital CMOS-based architecture designs originates from the massive redundant off-chip weight access during the numerous DNN inference in searching;
- We propose a design alternative that conducts in-situ One-Shot NAS searching, which can help fundamentally eliminate the massive redundant off-chip memory access during searching;

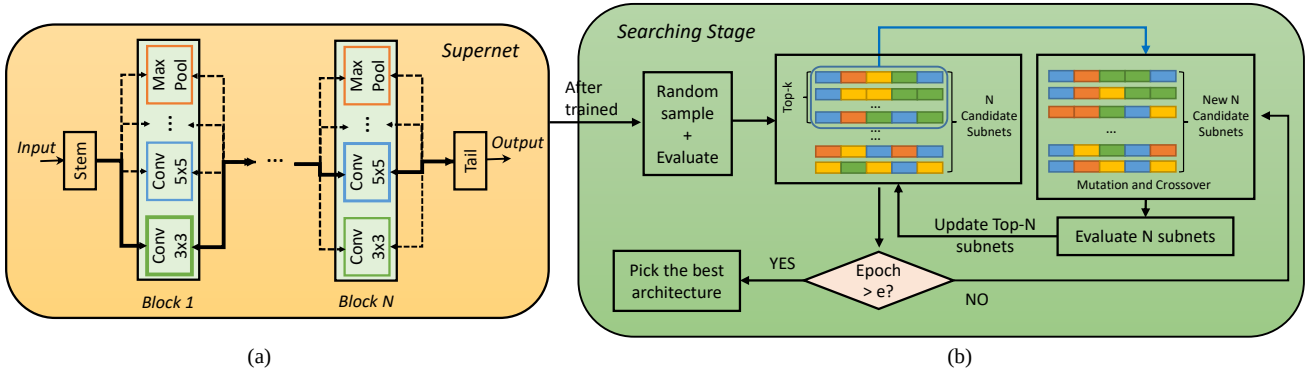


Figure 1: Illustration of One-Shot NAS. (a) An example of supernet structure in One-Shot NAS. (b) The evolutionary algorithm in One-Shot NAS searching stage.

- We reveal two major challenges of directly applying searching on the existing ReRAM accelerators, which can lead to severe contention and resource under-utilization;
- We propose the first highly-efficient in-situ One-Shot NAS search engine, named *NAS-SE*, for both single-device and large-scale deployment. *NAS-SE* meticulously addresses the pipeline contention and resource under-utilization challenges while successfully mitigating accuracy impact;
- Extensive evaluation on three state-of-the-art One-Shot NAS methods demonstrates *NAS-SE*'s far superior searching capability over the state-of-the-art designs.

2 BACKGROUND OF ONE-SHOT NAS

To improve search efficiency, *One-Shot NAS* approaches [2–4, 11, 13, 16, 56] have been introduced, which require to train a *Supernet* only once. Specifically, the optimization of weight and architecture is decoupled into two stages: training and searching. During the training stage, a supernet is trained until it achieves a high accuracy for a specific task. During this process, only weights are optimized. During the searching stage, a number of candidate subnet architectures are then sampled from the supernet, and evaluated on the validation dataset using the weights inherited from the supernet. This process will be repeated until an architecture with the highest validation accuracy is found. The key success of the One-Shot NAS is that the accuracy of a subnet architecture **A** using the inherited weights is highly predictive for the accuracy of **A** that is fully trained [16].

Supernet Training Stage Figure.1 (a) depicts a typical supernet structure. The backbone of a typical supernet is stacked by a number of blocks. Each block contains multiple design choices and only one will be selected during NAS. In Fig.1 (a), different design choices are represented by different colors. Various neural network operations, such as 3×3 or 5×5 convolutional layers or max pooling layer, can be chosen. In practice, one choice may contain a sequence of operations, e.g., a convolutional layer, a batch normalization layer and a ReLU function. A candidate subnet is then created by making a separate selection for each block.

Searching Stage As shown in Figure 1(b), the searching stage starts after the supernet is fully trained. A pool of candidate subnets

are initialized by sampling N (e.g., $N = 50$) candidate subnets randomly from the supernet. The weights in the supernet are directly passed to the candidate subnets, which are then evaluated on the validation datasets and ranked based on accuracy. In Figure 1(b), each subnet in the candidate pool is represented by a long horizontal bar comprised of multiple blocks. The color of a block indicates the block's choice. Once the pool is initialized, an *evolutionary algorithm* [14, 18, 22, 26, 27] is used to aid in the search for the best architecture. During each search epoch, the top- K (e.g., $K = 10$) architectures will be selected and evolved by performing mutation and crossover operations on them. These operations randomly perturb the blocks' choices in the top- k architectures and generate N subnets as new candidates. To update the candidate pool, the new candidate subnets are evaluated and ranked alongside the previous candidates. The preceding process is repeated until a certain number of epochs are reached, at which point the architecture with the highest accuracy is chosen from the candidate pool.

3 UNIQUE CHALLENGES FOR REAL-WORLD DEPLOYMENT

3.1 Major Bottleneck for One-Shot NAS

For the real-world deployment of One-Shot NAS, given the same machine learning task, the optimal architectures corresponding to different hardware devices may turn out to be very different due to the diverse hardware constraints such as inference latency, the required FLOPS achieved by the searched network architecture, etc. In general, the current One-Shot NAS methods can take up to 13 GPU days [16] to find an optimal architecture for a specific hardware device, with 12 GPU days spent on supernet training and 1 GPU day spent on searching. Although training a supernet is time-consuming, it is a one-time task. The post-trained supernet can be reused repeatedly during the searching stage to find optimal architectures for various hardware devices. Compared to the numerous searching scenarios, the training cost itself is often significantly amortized [4], which is one of the major reasons that One-Shot NAS becomes popular in the field of AutoML. Figure 2 depicts the breakdown of total cost (in terms of GPU days) in three state-of-the-art One-Shot NAS methods, namely SPOS [16], FairNAS [11], and GreedyNAS [56], as the number of target hardware platforms increases from ten to one hundred. As can be seen,

it takes 76.8 GPU days to *search* for the optimal architectures for the 100 targeted hardware platforms, accounting for an average of 88% of the overall cost. As a result, when applying One-Shot NAS approaches to various deployment scenarios, the cost of the searching phase is the primary bottleneck [4, 29].

3.2 Challenges for Efficient One-Shot NAS Searching on Digital CMOS Architectures

As mentioned in Sec.2, the One-Shot NAS searching stage iteratively performs inference on various sampled subnets and rank them according to the accuracy. The numerous neural network inferences clearly account for the majority of the search cost. For example, if each generation, i.e., a search epoch, has 50 candidate subnets ($N = 50$), and there are 20 search epochs in total, it is necessary to evaluate 1000 candidate subnets in order to find the optimal architecture for each deployment scenario. In the real-world One-Shot NAS deployment, the optimal models for diverse hardware devices are different because they have to meet different hardware efficiency constraints, including latency, energy, etc. For example, when a large amount of users simultaneously download an application from the AppStore, it requires to perform concurrent model searches for them based on their devices, which can be from iPhone 13 Pro with 16-core neural engine to old iPhone 6. Even for similar devices, the optimal model may still differ due to the latency demand, battery condition or workloads. Given a large number of deployment scenarios, e.g., 100, we must evaluate 100k subnets in total. When evaluating 100k subnets on digital CMOS-based architectures such as general-purpose GPUs and customized accelerators, we discovered significant redundancy in terms of weights' off-chip memory access.

Specifically, the architectures and weights among all the candidate subnets exhibit great similarity. For instance, in Figure 1, all the candidate subnets are drawn from the same post-trained supernet and the number of block options is typically limited, e.g., only 4 ~ 7 in the three benchmarks. Assume we need to randomly sample 100k subnets from the supernet, which has four choices per block. For each block n , 100k subnets will select a choice among four given choices so there is always one choice shared by at least 25k subnets, while other choices are also shared by thousands of subnets. These numerous blocks with shared choices are referred to as *repeated blocks*. Because a subnet's weights are directly inherited from the supernet, the weights of repeated blocks are exactly the same, and thus the 100k candidate subnets share a massive amount of weights. Since the same block weights in the supernet can be requested by different subnets and fetched thousands of times from the off-chip memory, evaluating these subnets sequentially on the current GPUs or DNN accelerators will result in massive redundant off-chip weight access.

Figure 3 shows the redundant off-chip memory access of weights in a certain block of a candidate subnet as the number of deployment scenarios increases. The searching process is carried out on a Nvidia Telsa P100 GPU. For each NAS benchmark, the basic experimental methodology is that we randomly select a block choice and calculate the overall off-chip memory access of its weights until the searching is completed for all the hardware platforms. For example, we randomly selected the weights of the 2nd choice of the

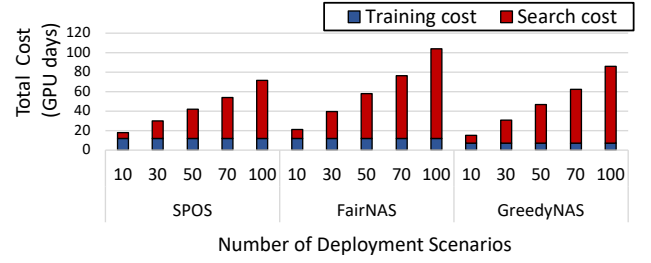


Figure 2: The breakdown of total costs when the number of deployment scenarios increases.

8th block in SPOS, denoted as W_2^8 ; the weights of the 2nd choice of the 14th block in FairNAS, denoted as W_2^{14} ; and the weights of the 2nd choice of the 15th block in GreedyNAS, denoted as W_2^{15} . As can be seen in Figure 3, the redundant memory access of W_2^8 reaches to 8.5GB when the number of hardware platforms increases to 100 for SPOS (i.e., 100K subnets to be evaluated in total), while the size of W_2^8 itself is only 0.34MB. The similar results can be observed in the other two benchmarks. Such massive redundant off-chip memory access is mainly caused by the repeated weight access across the subnet evaluation processes.

Recently, NASA [29] was proposed to reorganize the evaluation processes of the candidate subnets by executing the blocks that belong to different subnets at the same time. This enables NASA to fuse the redundant blocks and exploring temporal data reuse. However, their approach mainly focuses on the searching procedure within one generation and neglects the redundancy across a number of deployment scenarios in a common large-scale deployment scenario. Moreover, the proposed fusing technique only saves the weight memory access in the first few blocks of the candidate subnets, whereas the data reuse approach takes advantages of the weight reuse at the expense of input/output reuse. Figure 3 shows the redundant weight access after applying NASA's fusing and data reuse techniques in the GPU implementation of the One-Shot NAS searching. It should be noted that NASA's data reuse method allows for weight reuse while causing corresponding input/output data to be transferred between on-chip and off-chip memory. For a fair comparison between NASA and the GPU baseline case, the intermediate data for the off-chip memory access of one block is empirically limited to 117MB during one searching process, which is comparable to the redundant weight access of one block per searching. With the constraint above, we observed that an average of 32% redundant weights' off-chip memory access is reduced by NASA. However, this also incurs 117MB intermediate data access per SPOS search, and 105.4MB per FairNAS and GreedyNAS search, respectively. Therefore, the existing approaches are not adequate to fundamentally address the large volume of redundant weights' off-chip memory access.

Design Alternative. To deal with the significant memory overheads caused by the significant redundant weights, in-memory computing solutions seem to be a natural design choice. One can "One-Shot" store all of the weights of the post-trained supernet into ReRAM and perform in-memory network evaluation on the ReRAM for the candidate subnets. All of the previously mentioned redundant off-chip weight access will be eliminated by moving

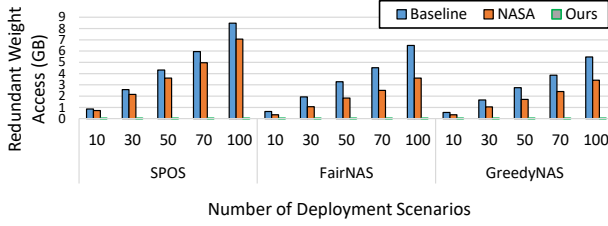


Figure 3: The off-chip memory access of redundant weights in a certain block of a subnet over the number of deployment scenarios for the three One-Shot NAS benchmarks.

the searching stage to ReRAM (Figure 3). Because weight reuse is not required when performing inference on ReRAM, the input image can be processed one at a time, i.e., batch size = 1, and the intermediate data can be small enough to fit onto an on-chip buffer or eDRAM. As a result, this solution has the potential to reduce the off-chip data transfer of the intermediate feature maps as well. The key to this design alternative’s feasibility is based on two important algorithmic features of the One-Shot NAS. First, the supernet is only trained once and can be reused in the searching stage for different target hardware platforms. Second, each candidate subnet can be obtained by selecting a path from the supernet, and its weights can be directly inherited.

4 CHALLENGES FOR DESIGNING EFFICIENT RERAM-BASED SEARCHING

Although the naive solution above essentially solves the weight transfer problem, we discover two major challenges of practically deploying the searching process on ReRAM: *resource contention* and *resource undertutilization*.

4.1 Resource Contention

After the supernet is fully trained, all of its weights are stored in the ReRAM. When the search initiates, it appears inefficient to only evaluate one subnet at a time because other weights stored on the ReRAM are not used. For instance, given a supernet with 4 choices per block, only 1/4 weights will be used when evaluating a single subnet at a time. This will cause 3/4 of the crossbars become idle since the weights of all the choices (i.e., the weights of the entire supernet) are already stored in ReRAM. To improve the utilization of crossbars, one can run a group of subnets in parallel. Note that the group size G should be no more than the total number of choices per block, which is determined by the supernet structure. Comparing with running a single subnet at a time, we can reach an ideal speedup of G when running G subnets at a time. However, such ideal speedup can not achieved due to the severe resource contention problem, which is depicted in Figure 4. As shown in the upper-left corner of the figure, there are three candidate subnets to be evaluated in parallel on the ReRAM. Each circle in the subnets is a block and the letter within the circle represents the block’s choice. In this example, we assume that each block in the supernet has three choices, namely a, b, and c. The inference of these networks will be performed on the ReRAM tiles, each of which is assumed to store all the weights of a block’s choice, i.e., w_{Ch}^B . Each w_{Ch}^B produces its corresponding intermediate outputs O_n^B during the inference. Here Ch , B and n denote the choice of the block, the position of the block

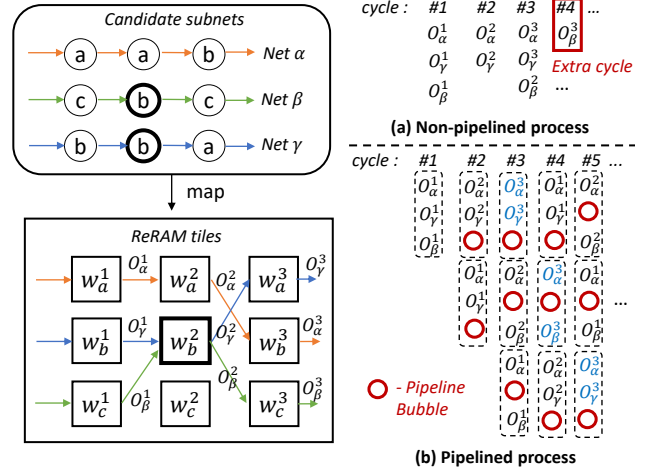


Figure 4: Resource contention issues when executing multiple network evaluations on ReRAM in (a) non-pipelined and (b) pipelined manner.

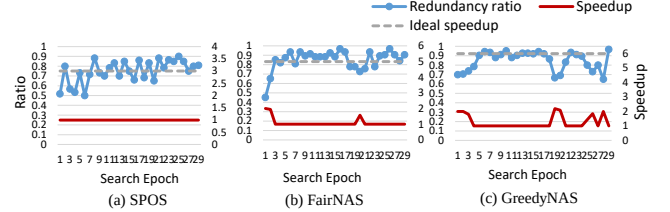


Figure 5: The ratio of conflicted weight blocks, ideal speedup and actual speedup vs searching epochs, when running multiple subnets in parallel on the ReRAM.

in the subnet and the network, respectively. As highlighted in the bolder line, Net 2 and Net 3 have the same choice in the 2nd block, which means their weights in the 2nd blocks are the same. In this case, the outputs O_β^1 and O_γ^1 require the same weights w_b^2 (e.g., O_γ^2 and O_β^2 both require the contended weights from w_b^2) at the same time, in order to perform computations in the next block. Unlike the computation pattern in the digital CMOS-based architecture, the contended weights w_b^2 can not be shared by multiple inputs at the same time unless we duplicate w_b^2 on the crossbars. However, it’s impractical to duplicate in this way since any group of weights may be contended during the dynamic evaluation process of the evolution algorithm. Therefore, the computation of the 2nd block for Net2 and Net 3 have to be processed in sequence. Such phenomenon is defined as *resource contention*.

To investigate the seriousness of this resource contention issue, at each search epoch, we randomly select a group of candidate subnets from N (e.g., $N=50$) candidates subnets, with the group size of 3, 5, and 6 for SPOS, FairNAS and GreedyNAS, respectively. Figure 5 shows the ratio of conflicted weight blocks (the blocks share the same choice) and speedups in three NAS benchmarks when evaluating the selected group of subnets in parallel. The conflict ratio is calculated as $num_repeated_blocks/num_total_blocks$ in the group of subnets, which is 2/9 in the example shown in Figure 4.

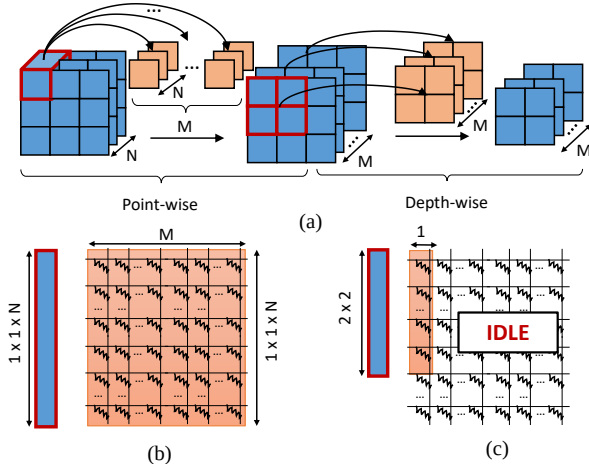


Figure 6: (a) The computation pattern of the depth-wise separable convolution in each block and how (b) point-wise convolution and (c) depth-wise convolution are mapped to the ReRAM crossbar.

As shown in Figure 5, as the searching proceeds, we observe that the ratio of the conflicted blocks (left-axis) quickly increases and remains at a high level, i.e., 80% ~ 90%. The conflicted blocks will impact the speedup (right-axis) achieved by running a group of subnets at a time. Based on the analysis for Figure 4 (b), we built a performance model that considers the resource contention issue, we applied it to calculate the actual speedup given a group of randomly selected subnets. As can be seen in Figure 5, the actual speedup is around 1 for the three benchmarks across search epochs. This occurs when all the subnets in a group share the same choice for a certain block (i.e., fully conflicted), at which point only $1/G$ (G is the group size) throughput is achieved after the pipeline is filled. Overall, because of this severe resource contention issue, processing multiple subnets in parallel results in an average speedup of only $1.11\times$, which is far below than the ideal speedup.

4.2 Resource Under-Utilization in Depth-wise Convolutional Layers

The network structure of the building block is predefined in the existing One-Shot NAS approaches. For instance, current One-Shot NAS methods adopt *depth-wise separable convolution* as a macro network structure for each building block (Figure 1). This is because depth-wise separable convolution has demonstrated excellent accuracy with significantly lower computation requirements [10] for computer vision tasks, and it is widely adopted in different cutting-edge CNN models [10, 19, 28, 33, 39, 41, 42]. Each block's choice varies in structural hyperparameters like kernel size or channel expansion ratio. For example, FairNAS and GreedyNAS apply a MobileNetV2 inverted bottleneck [33] as the basic building block, with a depth-wise separable convolution as its main component.

Figure 6 shows an example of performing depth-wise separable convolution on an input of size $3 \times 3 \times N$ and how the computations are mapped to a ReRAM crossbar in the conventional mapping method [21, 34, 54]. We assume input and weights are both in 16-bit

precision, and each ReRAM cell stores two bits and the Digital-to-Analog Converter(DAC) resolution is 1 bit. The depth-wise separable convolution consists of two steps: point-wise convolution (PW) and depth-wise convolution (DW). For the PW layer, an input of size $1 \times 1 \times N$ is convolved with M filters in each cycle, each of which has the size of $1 \times 1 \times N$. As shown in Figure 6 (b), M filters will be mapped to M columns on a ReRAM crossbar and each filter of size $1 \times 1 \times N$ will be flattened into a vector. For PW, each input can be multiplied with M weights on the ReRAM at the same time. Since M, N are usually large, e.g., $32 \sim 320$ in SPOS, a ReRAM crossbar of a typical size of 128×128 can be easily fully utilized. For a DW layer, in each cycle an input of size $2 \times 2 \times 1$ only convolves with its corresponding (in the same channel) 2×2 weight kernel. Since there is no input sharing across different filters, one input only multiply with one weight at a time. Therefore, only $16/2 = 8$ columns (one weight occupies 8 columns) are utilized on a 128×128 crossbar. For each supernet block, the kernel size of the DW is within 3, 5, 7, thus the length of the input vector is limited within $9 \sim 49$, which also leads to the under-utilization problem in the row dimension. Overall, only 0.43% ~ 2.3% ReRAM cells can be activated at the same time when executing a DW layer of a subnet if using the conventional mapping method.

When evaluating the subnets on the existing ReRAM-based DNN accelerators, such resource under-utilization can directly impede peak performance and computation efficiency. In ISAAC [34], for example, the pipeline design requires the output neurons across the output channel to be produced at the same time in order to satisfy the next layer's read request. However, when the DW layer of One-Shot NAS is executed, it either breaks the pipeline by requiring many more cycles to obtain the outputs (if different kernels are mapped to the same crossbar and activated only one at a time), or sacrifices chip area which reduces computation/power/storage efficiency (if different kernels are mapped to different crossbars).

5 NAS-SE METHODOLOGY

5.1 Network Diversification

As introduced in Sec.4.1, it is extremely inefficient to perform inference for a group of candidate subnets in parallel on ReRAM. The fundamental reason is that many blocks' choices are shared by different candidate subnets in the group, causing different subnets to compete for the same weights stored in the ReRAM. To address this problem, we made an observation that during the evaluation process, all the candidate subnets in the current search epoch are obtained through random perturbation of blocks' choices, namely mutation and crossover, on the top- K (e.g., $K=10$) subnets in the previous search epoch. As shown in Figure 7(a), a mutation operation randomly selects a block with a mutation rate and randomly alters the block's choice. Crossover operation randomly selects a choice for each block from the the corresponding blocks' choices of the two parent subnets. Considering such randomness of the blocks' choices in the candidate subnets, if we further forcibly alter the choices in the current candidate subnets, it simply introduces randomness to the original random process. Based on this insight, we first propose a brutal solution that forces a group of subnets which are evaluated in parallel, to select different choices for the same block. We define this solution as *network diversification* and

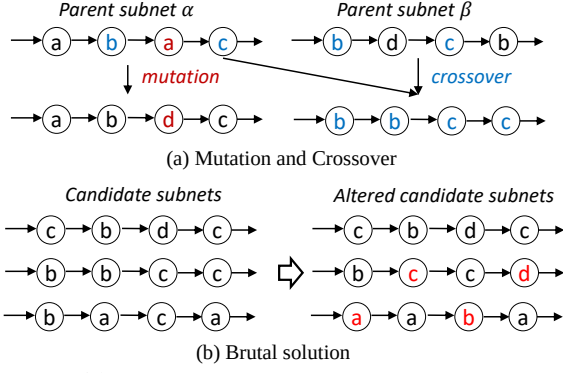


Figure 7: (a) The operations in mutation and crossover. (b) Brute-force approach: forcibly diversifying the blocks' choices of the candidate subnets.

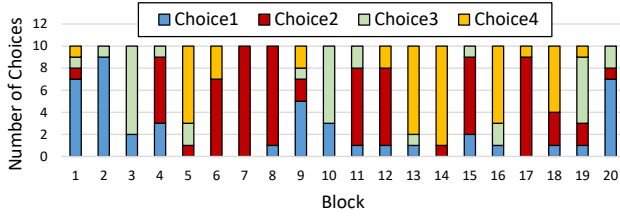


Figure 8: The blocks' choices of 10 child subnets that originate from the same parent subnet at the 1st search epoch.

show an example in Figure 7(b). With such meticulous network diversification, there will be no conflicts of blocks' choices within a group of subnets, thus addressing resource contention.

However, this naive network diversification often hinders the search process to find the optimal architecture because it can significantly deviate its identified architecture from the top-K subnets' architectures ranked by the core search algorithm. Specifically, as the searching proceeds, the blocks' choices in the top-K architectures become closer and closer to the optimal choices. Brute-force changes on too many near-optimal choices will prevent the evolutionary algorithm from finding better architectures. In order to reduce the accuracy loss brought by the brute-force solution, we propose a two-level hierarchical scheduling strategy that significantly mitigates the issue of blocks' choices conflicts, so that *much fewer blocks' choices need to be altered* when applying network diversification. We first investigate the scheduling problem during the searching process for a single target device, and propose an intra-search grouping strategy. Then we leverage the potential opportunities when the overall task is extended to a large number of deployment scenarios, and propose an inter-search grouping strategy to further mitigate the resource contention issues.

5.1.1 Intra-Search Grouping Strategy. In order to reduce the block conflicts within a subnet group, a dynamic grouping strategy is demanded to form a group where the subnets have as many different blocks' choices as possible. It's clear that using a traditional clustering algorithm is time-consuming and can incur significant latency overheads since the grouping process is required at every search epoch. Therefore, a more efficient and effective grouping strategy should be proposed. Interestingly, we observe that the child subnets that are generated through mutation(crossover) by

the same parent subnet(s), defined as *siblings subnets*, often exhibit high similarity. Such similarity is due to two main reasons. First, in the One-Shot NAS searching process, the mutation rate is normally low, i.e., 0.1. Second, the crossover simply selects one of the choices from the parent subnets and does not apply any further changes. As shown in Figure 8, we investigate the blocks' choices of the siblings subnets that originate from the top-1 parent subnet at search epoch 1 in the SPOS benchmark, which has 20 blocks with 4 choices per block. The top-1 parent subnet has 10 child subnets in total as it was randomly selected 10 times. For each block, we count the number of appearances for each choice in these 10 child subnets. The results show that there is always a common choice shared by most of the siblings subnets, which indicates that the block conflicts between siblings subnets can be serious.

Based on this observation, we propose an intra-search grouping strategy that, instead of finding those diverse subnets and grouping them, we separate the similar siblings subnets into different evaluation groups, which indirectly maximizes the diversity degree of the subnets in each evaluation group. Figure 9(a) shows an example of using our intra-search grouping strategy. Assume the size of the evaluation group is 3, on the left of the figure, the candidate subnets are evolved from the parent subnets $p1 \sim p10$ via mutation and crossover. Without our grouping strategy, the evaluation groups could be formed as the red boxes, which causes serious block conflicts because most of the subnets in the group are siblings. The proposed grouping strategy allocates the sibling subnets into different groups so that most of the subnets within an evaluation group originate from different parents. In order to implement this strategy, we only need a P_ID to record the index of each candidate subnet's parent(s).

5.1.2 Inter-Search Grouping Strategy. As introduced in Sec.3.1, each deployment scenario requires a unique searching process to find its optimal architecture according to its hardware constraints. The total number of computations of a network architecture (FLOPs) is commonly used as a constraint in the current One-Shot NAS methods. For example, the searching process for a resource-constrained device will limit the FLOPs of the candidate subnets to be smaller than, e.g., 290 million FLOPs, while for a more powerful device the FLOPs limitation could be set to 430 million FLOPs. A subnet's FLOPs are directly affected by the choice in each block. For example, for the 1st block in the FairNAS supernet, choosing a 7×7 residual block with the channel expansion ratio = 6 results in $3.4 \times$ computation compared to choosing a 3×3 residual block with the channel expansion ratio = 3. At the late searching stage, the evolutionary algorithm already finds several good architectures that steadily appears in the Top-k architectures for every searching process. We observed that the good architectures from *different searching process* demonstrate higher diversity than those from the same searching process. This is because the subnet's architecture of the resource-constrained device tends to select the choice with fewer computations, which creates a diversity in block choices for more powerful devices. Note that such diversity only appears between the good architectures in the late search stage, and it's barely observed between the chaotic architectures in the early search stage.

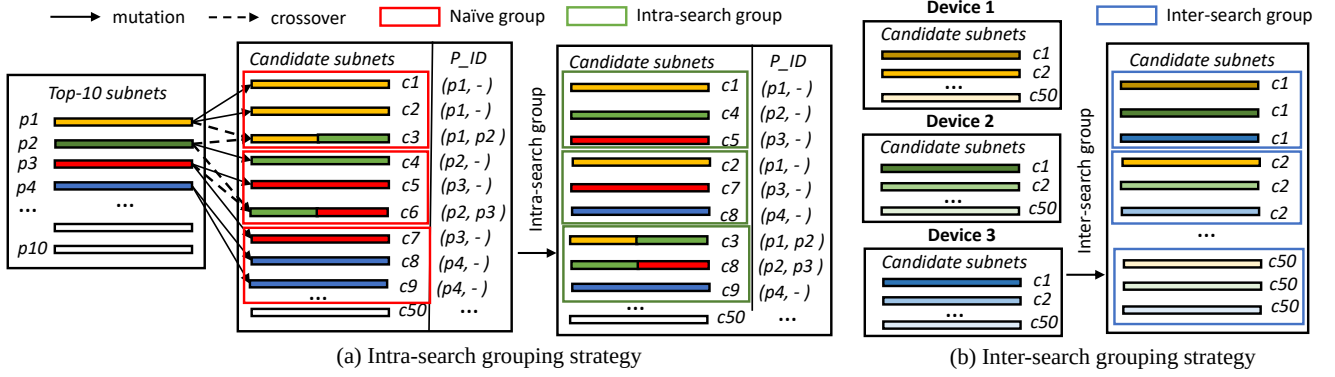


Figure 9: (a) Intra-search grouping strategy: separating the siblings subnets into different groups. (b) Inter-search grouping strategy: evaluating candidates subnets from different deployment scenarios in parallel.

Based on this insight, we propose an inter-search grouping strategy that applied in the late search stage, which is shown in Figure 9(b). Inter-search grouping strategy allocates the candidate subnets from different searching processes into an evaluation group, which enhances the network diversity in the group and reduces the number of blocks' choices that need to be forcibly altered. The subnets in an evaluation group will be evaluated in parallel on the ReRAM. In order to achieve a higher network diversity in the inter-search grouping strategy, we propose to schedule the searching tasks for different hardware devices at the beginning of the search stage. The sequence of the searching tasks will be organized in a way to ensure that there will be a gap of FLOPs constraints between neighbour searching tasks. Since we do not require the scheduling operations to achieve the optimal diversity, the overheads of such scheduling are insignificant.

5.2 Novel Mapping for Depthwise Convolution

To address the resource under-utilization problem during the execution of the depth-wise convolutional layers, we propose a novel mapping scheme that can significantly improve the utilization by exploring the *kernel-level input activation reuse*. Recall that in the conventional mapping scheme, the input activation can not be shared by multiple weights along the crossbar column, since the input activation only multiply with the weights in the corresponding channel. In our novel mapping scheme, we exploit the input activation reuse in a finer granularity, namely kernel-level input reuse, to improve the resource utilization on a crossbar.

As shown in Figure 10, the computation workloads of M output channels are distributed across M crossbars. Each crossbar in the PE is responsible for computing the output neurons in an output channel. Thus, the computations on a crossbar are convolving an input feature map with its corresponding weight kernel. Noticing that a certain input activation may multiply with several weights in the kernel; we flatten the feature map as a 1-D vector and duplicate the weights along the column to achieve the input reuse, which is shown on the left side of the Figure 10. The duplicated weights are placed on the proper places so that they can correctly multiply with the corresponding inputs. In this example, since input activation a_5 will be convolved with all the weights in the kernel, $w_1 \sim w_4$ are placed in the first row. For activations that locate at the edge (a_2, a_4, a_6, a_8) or the corner (a_1, a_3, a_7, a_9) of the feature

map, since they only multiply with some of the weights in the kernel, we store zero values in the corresponding ReRAM cells. Using this mapping scheme, 4 bitlines (columns) can produce 4 output neurons in one cycle, which is $4\times$ faster than the conventional mapping scheme. This speedup comes from the improvement of resource utilization. Assume the output tile size is $T_r \times T_c$, the utilization along the column dimension can be potentially increased by $T_r \times T_c$. Since the output size $R \times C$ is no smaller than 7×7 in the depth-wise convolution layers of the One-Shot NAS supernet, the convolution can always occupy at least $7 \times 7 \times \frac{16}{2} = 392$ columns, which is adequate to fully utilize three 128×128 crossbars. Assume the tile size of input activations is $T_h \times T_w$ and the kernel size is $k \times k$, the utilization along the row dimension is also increased by $\frac{T_h \times T_w}{k \times k}$. Such utilization improvement enables much less crossbars required to achieve the same throughput. For instance, assume we pair the pipeline design in ISAAC [34] with the conventional mapping scheme shown in Figure 6, the convolution operation shown in Figure 10 may require four duplicated crossbars to compute four output neurons in parallel to fulfill the data request for the next layer. Obviously, this design suffers from low hardware utilization. In our proposed mapping scheme, we only require one crossbar to produce four output neurons at a time with high utilization level, which significantly reduces the area overheads and improve the computation efficiency.

6 IMPLEMENTATION OF NAS-SE

Architecture Overview. Figure 11 shows an overall diagram of the NAS-SE search engine. There are two major components in NAS-SE: a ReRAM-based DNN accelerator that evaluates the accuracy of candidate subnets and a search unit that is responsible for scheduling the two-phased searching processes given a number of searching tasks. The ReRAM-based DNN accelerator mainly consists of a number of ReRAM tiles that stores all the weights of a supernet. The tiles are virtually divided into a number of choice areas, each of which contains several tiles that store all the weights of a block choice. The inference results are temporarily stored into an accuracy recorder to obtain the final network evaluation results, as highlighted by the blue box.

Tile Design. In the NAS-SE design, multiple subnets are evaluated simultaneously on the ReRAM and each subnet can have different choices for different blocks. For each subnet, the outputs

Table 1: NAS-SE hardware characteristics

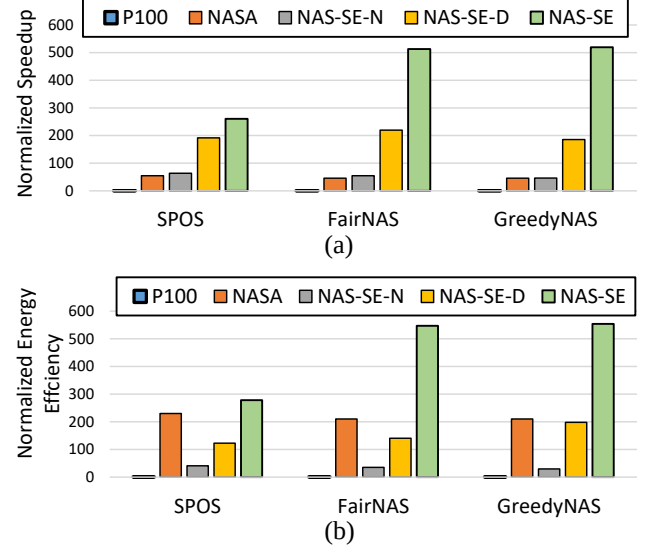
Component	Spec	Power <i>mW</i>	Area <i>mm²</i>
PE configurations, 12 PEs per tile			
ADC	number:8; 8-bit resolution; 1.2GSps	16	0.0096
DAC	8 x 128; 1-bit resolution	4	0.00017
S+H	8 x 128	0.01	0.00004
Crossbar	8 x 128 x 128; 2 bits/cell	2.4	0.0002
S+A	number: 4	0.2	0.00024
IR	size: 2KB	1.15	0.0016
OR	size: 256B	0.26	0.0009
IR reorder unit	number: 1	2.02	0.0032
Tile configurations, 684 tiles per chip, 1.2GHz			
eDRAM buffer	64KB, 4 banks, 256 bus width	20.7	0.083
Memory bus	384 wires	7	0.09
Router	flit size: 32; 8 ports	42	0.151
ReLu unit	number: 1	0.14	0.0002
S+A	number: 1	0.05	0.00006
Pooling	number: 1	0.4	0.00024
OR	3KB	1.82	0.0036
Multiplexer	number: 1	0.17	0.00024
PE	number: 12	312.48	0.1957
Searching unit	number: 1; total buffer size: 10KB	5.9	0.0145
Accuracy recorder	number: 1; total buffer size: 128B	0.078	0.0014

One-Shot NAS benchmarks We evaluate NAS-SE on three state-of-the-art One-Shot NAS methods: SPOS [16], FairNAS [11], and GreedyNAS [56]. The supernet training and architecture searching stages of all the benchmarks are conducted on ImageNet datasets [12]. All the supernets are trained in FP32 precision. The parameters in the One-Shot NAS algorithm are kept the same as the original papers. Table 2 summarizes the detailed configurations of three benchmarks, including the structure of all the blocks' choices. For example, among 4 choices in a SPOS building block, three choices adopts a typical structure from ShuffleNet_V2 with different kernel size (k) selected from $\{3, 5, 7\}$. The other choice adopts the structure of ShuffleNet_Xception. In FairNAS, 6 choices adopts Inverted Bottleneck structure with different kernel size(k) and channel expansion ratio (r). All the building blocks mainly consist of point-wise(pw) and depth-wise(dw) convolution layers.

Table 2: Configurations of One-Shot NAS benchmarks.

Methods	#Blocks	#Choices	Choices	Structure
SPOS	20	4	ShuffleNet_V2 $k = \{3, 5, 7\}$	$pw - dw - pw$
			Shuffle_Xception	$dw - pw$
FairNAS	19	6	Inverted Bottleneck $k = \{3, 5, 7\}$ $r = \{3, 6\}$	$pw - dw - pw$
GreedyNAS	21	7	Inverted Bottleneck $k = \{3, 5, 7\}$ $r = \{3, 6\}$	$pw - dw - pw$
			Identity block	-

Comparison cases To demonstrate the effectiveness of NAS-SE, we compare it with four hardware designs: Firstly, we use a CPU-GPU system as the baseline case. The CPUs are Intel Xeon CPU E5-2650 v2 processors that operates at 2.60GHz, and the GPU is a NVIDIA Tesla P100 GPU with 9.3 TeraFOPS/s peak performance and 732GB/s memory bandwidth. The overall One-Shot NAS algorithm is implemented using Pytorch [32]. Secondly, we compare with NASA, a customized accelerator designed for NAS searching process. Thirdly, to test the effectiveness of our network diversification and novel mapping scheme, we compare with a naive ReRAM-based search engine, i.e., NAS-SE-N, that does not apply these strategies. Further, to breakdown the benefits brought by our techniques, we compare with a more advanced ReRAM-based

**Figure 13: Comparison of speedup and energy efficiency between NAS-SE and other designs.**

search engine, i.e., NAS-SE-D, which employs the network diversification technique. For NAS-SE-N, NAS-SE-D and NAS-SE, we use the same basic hardware configurations for fair comparison. Compared with NAS-SE-N, NAS-SE-D only adds hardware modules for intra- and inter-search grouping strategies. Thanks to the novel mapping in DW layers, NAS-SE design has much fewer crossbars than other two NAS-SE variants.

7.2 Evaluation Results

Performance and Energy The area and maximum power of NAS-SE chip are $282.31mm^2$ and 241.6W, respectively. The computation efficiency and power efficiency of NAS-SE are $113.86GOPS/(s \cdot mm^2)$ and $132.76GOPS/(s \cdot W)$.

Figure.13 (a) shows the performance speedup of NAS-SE compared against GPU baseline and other designs. As it shows, by moving the searching stage on ReRAM, even the naive design, NAS-SE-N, achieves an averagely $55\times$ and $1.12\times$ performance improvement compared with P100 and NASA, respectively. The main factors that limit the performance of NAS-SE-N are the serious resource contention and under-utilization issues, as introduced in Sec.4. When we apply the network diversification along with intra-search and inter-search grouping strategies, a more advanced design, i.e., NAS-SE-D, achieves an averagely $199\times$ (up to $220\times$) and $4.06\times$ (up to $4.78\times$) performance improvement over P100 and NASA, respectively. The reason of the significant performance improvement is that our network diversification approach enables multiple candidate subnets to be inferred in parallel on ReRAM. Finally, our proposed design, NAS-SE, which equipped with both network diversification and novel mapping scheme, further boosts the performance speedup to averagely $431\times$ and $8.8\times$ over P100 and NASA. Thanks to the novel mapping scheme, the ReRAM crossbar on the NAS-SE enjoys a much higher utilization level, which is convert to averagely $2.17\times$ speedup over NAS-SE-D. For different benchmarks, the performance gain achieved by NAS-SE can be

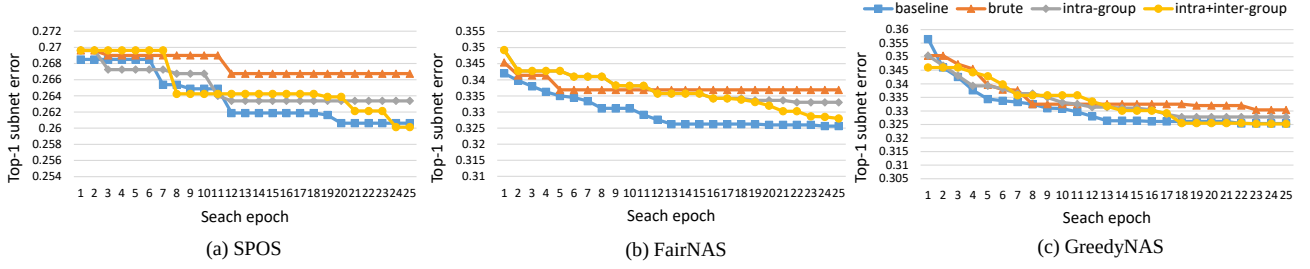


Figure 14: The evaluation accuracy of top-1 architecture for (a) SPOS, (b) FairNAS, and (c) GreedyNAS during the searching stage

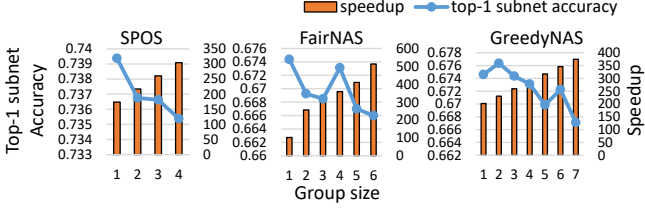


Figure 15: The speedup and top-1 architecture's accuracy v.s. the group size of intra-search strategy.

different. For example, NAS-SE achieves best performance improvement over NASA, i.e., 11.3 $\times$, on GreedyNAS while achieves only 4.8 $\times$ on SPOS. This is because the weights of SPOS supernet does not fully occupy the chip, thus a large number of tiles are not used.

Figure.13 (b) shows the energy efficiency of NAS-SE compared with other designs. The energy efficiency is defined as throughput per watt (GOPS/Watt). As can be seen, NAS-SE-N and NAS-SE-D are 36.6 $\times$ and 155.3 $\times$ more energy efficient than P100 but they both have worse energy efficiency compared with NASA. This is because a majority of crossbars on these two engines are significantly under-utilized ($\leq 1\%$) due to the inefficient conventional mapping scheme. As a result, although both design can reach a considerable throughput, they require much more number of crossbars to accomplish it. Nevertheless, by introducing the novel mapping scheme, NAS-SE achieves an averagely 446 $\times$ (up to 537 $\times$) and 2.05 $\times$ (up to 2.55 $\times$) energy efficiency compared with P100 and NASA.

Searching Accuracy As mentioned in Sec.5.1, forcibly altering the blocks' choices could deviate the search process from finding the optimal architecture. To test the our intra-search and inter-search grouping strategies' abilities to resist the searching deviation, we extract the top-1 architecture's evaluation accuracy in each search epoch when using different search strategies. As shown in Fig.14, when using the brute-force altering method, the top-1 architecture obtained at 25th search epoch is averagely 0.75% less accurate than that using the baseline searching algorithm. After applying our intra-search and inter-search grouping strategies, the accuracy loss is reduced to averagely 0.05% for all three benchmarks.

Sensitivity Analysis Group size: While our proposed intra-search grouping strategy reduces the block conflicts during the early searching stage and allows a group of candidates subnets running in parallel, some block choices are still unavoidably altered, which could lead to the final search accuracy loss. To reduce the accuracy loss, a possible way is reducing the size of the evaluation group so that less subnets will run together, which may decrease the block conflicts. On the other hand, this will also degrade the performance speedup. Such trade-off motivates us to study the

impact of the intra-search group size on both performance and accuracy. Figure.15 shows the speedup of NAS-SE over P100 and the top-1 architecture's accuracy when we vary the group size in the intra-search strategy. As can be seen, the performance speedup increases linearly as the group size increases, while the accuracy shows a trend of decrease. For SPOS, FairNAS and GreedyNAS, we choose three sweetspots of group size, i.e., 3, 4, and 6, respectively, which achieves both high accuracy and performance.

8 DISCUSSION

Although one-shot NAS is originally proposed to automate the architecture design process of CNNs, recent works [6, 48] have shown some initial success when applying it to Vision Transformers (ViT) for CV tasks and small transformers (weight size less than 100M) for NLP. These works adopt the same methodology, i.e., supernet training, weight sharing and evolutionary search, from the CNN-based one-shot NAS works. Thus, their methods will cause the same issue given a large-scale deployment scenario, i.e. expensive searching processes due to the redundant memory access. Our ReRAM solution can effectively address this problem by storing the transformer's weights to the crossbar for once and then perform in-memory inference. Our software optimizations, i.e., inter- and intra-search grouping strategies, will also remain effective as they are designed mainly based on the characteristics of evolutionary search. Since the computational mechanism and the activation functions can be different between a convolution layer and an encoder/decoder layer, hardware modifications are required in order to extend NAS-SE to execute transformers. A recent ReRAM-based transformer accelerator [55] has shown the feasibility to execute transformers efficiently on ReRAM and their hardware design has minor conflict with NAS-SE. Investigating how we can modify our hardware to be more general for broader ML applications will be one of our future works.

9 RELATED WORKS

NAS acceleration NAS methods proposed in the early stage [62] are very time-consuming due to the massive training processes. From the algorithm perspective, [5, 24, 50] leveraged weight sharing strategy to reduce the cost of NAS. However, these methods still require a supernet training process for each searching task. To this end, One-Shot NAS methods [11, 16, 56] are proposed that significantly reduces the overall searching time for large deployment scenarios. To further reduce the cost of current One-Shot NAS methods, NASA [29] designed a specific DNN accelerator for efficient NAS searching. They proposed computation sharing and

data reuse techniques and achieved better performance compared with GPUs by using the on-chip mapping scheme. However, considering the large-scale deployment of One-Shot NAS, the searching efficiency are still required to be enhanced. Our work, for the first time, leverages the in-memory computing to greatly reduce the searching time and energy efficiency of One-Shot NAS methods.

PIM-based CNN accelerator PIM-based NN accelerators [7, 9, 34, 53, 54, 59] have achieved significant improvement of latency and energy efficiency on various machine learning workloads, which makes them become a competitor of digital-based accelerators [8, 25, 44, 45, 60]. [34] proposed a full-fledged ReRAM-based CNN accelerator with a novel pipeline design. Pipelayer [36] designed a batch-level pipeline process and supports both inference and training. [54] proposed a practical ReRAM-based DNN accelerator that utilizes the smaller operation units to exploit the data sparsity in DNN models. PUMA [1] uses ReRAM crossbars to build a general ISA-based programmable accelerator to accelerator various ML applications. The above accelerators are incapable of solving the resource contention and under-utilization problems during the One-Shot NAS searching processes. The design for sparse computations in [21, 49, 54] is orthogonal to our work.

10 CONCLUSION

In this paper, we reveal the challenges of deploying real-world One-Shot NAS on digital CMOS hardware and propose to port the searching stage to ReRAM crossbars. We propose network diversification and novel mapping techniques to tackle two key challenges of directly conducting search process on the ReRAM. Our experimental results show that NAS-SE achieves an average of $8.8\times$ and $2.1\times$ speedup and energy efficiency over the state-of-the-art digital-based customized NAS accelerator(NASA), respectively.

ACKNOWLEDGMENTS

This research is partially supported by NSF grants CCF-2130688, CCF-1900904, CNS-2107057, University of Sydney faculty startup funding, and Australia Research Council (ARC) Discovery Project DP210101984. Jing Wang is the corresponding author.

REFERENCES

- [1] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Geoffrey Ndu, Martin Foltin, R Stanley Williams, Paolo Faraboschi, Wen-mei W Hwu, John Paul Strachan, Kaushik Roy, et al. 2019. PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 715–731.
- [2] Gabriel Bender, Pieter-Jan Kindermans, Barret Zoph, Vijay Vasudevan, and Quoc Le. 2018. Understanding and simplifying one-shot architecture search. In *International Conference on Machine Learning*. PMLR, 550–559.
- [3] Andrew Brock, Theodore Lim, James M Ritchie, and Nick Weston. 2017. Smash: one-shot model architecture search through hypernetworks. *arXiv preprint arXiv:1708.05344* (2017).
- [4] Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. 2019. Once-for-all: Train one network and specialize it for efficient deployment. *arXiv preprint arXiv:1908.09791* (2019).
- [5] Han Cai, Ligeng Zhu, and Song Han. 2018. Proxylessnas: Direct neural architecture search on target task and hardware. *arXiv preprint arXiv:1812.00332* (2018).
- [6] Minghao Chen, Houwen Peng, Jianlong Fu, and Haibin Ling. 2021. Autoformer: Searching transformers for visual recognition. In *Proceedings of the IEEE/CVF international conference on computer vision*. 12270–12280.
- [7] Wei-Hao Chen, Kai-Xiang Li, Wei-Yu Lin, Kuo-Hsiang Hsu, Pin-Yi Li, Cheng-Han Yang, Cheng-Xin Xue, En-Yu Yang, Yen-Kai Chen, Yun-Sheng Chang, et al. 2018. A 65nm 1Mb nonvolatile computing-in-memory ReRAM macro with sub-16ns multiply-and-accumulate for binary DNN AI edge processors. In *2018 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 494–496.
- [8] Yunji Chen, Tao Luo, Shaoli Liu, Shijin Zhang, Liqiang He, Jia Wang, Ling Li, Tianshi Chen, Zhiwei Xu, Ninghui Sun, et al. 2014. Dadiannao: A machine-learning supercomputer. In *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 609–622.
- [9] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. Prime: A novel processing-in-memory architecture for neural network computation in rram-based main memory. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 27–39.
- [10] François Chollet. 2017. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1251–1258.
- [11] Xiangxiang Chu, Bo Zhang, and Ruijun Xu. 2021. Fairnas: Rethinking evaluation fairness of weight sharing neural architecture search. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 12239–12248.
- [12] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 248–255.
- [13] Xuanyi Dong and Yi Yang. 2019. One-shot neural architecture search via self-evaluated template network. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 3681–3690.
- [14] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2018. Efficient multi-objective neural architecture search via lamarkian evolution. *arXiv preprint arXiv:1804.09081* (2018).
- [15] Ross Girshick. 2015. Fast r-cnn. In *Proceedings of the IEEE international conference on computer vision*. 1440–1448.
- [16] Zichao Guo, Xiangyu Zhang, Haoyuan Mu, Wen Heng, Zechun Liu, Yichen Wei, and Jian Sun. 2020. Single path one-shot neural architecture search with uniform sampling. In *European Conference on Computer Vision*. Springer, 544–560.
- [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [18] Kary Ho, Andrew Gilbert, Hailin Jin, and John Collomosse. 2021. Neural architecture search for deep image prior. *Computers & Graphics* 98 (2021), 188–196.
- [19] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).
- [20] Rachel Huang, Jonathan Pedoem, and Cuixian Chen. 2018. YOLO-LITE: a real-time object detection algorithm optimized for non-GPU computers. In *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 2503–2510.
- [21] Houxiang Ji, Linghao Song, Li Jiang, Hai Li, and Yiran Chen. 2018. ReCom: An efficient resistive accelerator for compressed deep neural networks. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 237–240.
- [22] David Jones, Anja Schroeder, and Geoff Nitschke. 2019. Evolutionary deep learning to identify galaxies in the zone of avoidance. *arXiv preprint arXiv:1903.07461* (2019).
- [23] Patrick Judd, Jorge Albericio, Tayler Hetherington, Tor Aamodt, Natalie Enright Jerger, Raquel Urtasun, and Andreas Moshovos. 2015. Reduced-precision strategies for bounded memory in deep neural nets. *arXiv preprint arXiv:1511.05236* (2015).
- [24] Hanxiao Liu, Karen Simonyan, and Yiming Yang. 2018. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055* (2018).
- [25] Wenyan Lu, Guihai Yan, Jiajun Li, Shijun Gong, Yinhe Han, and Xiaowei Li. 2017. Flexflow: A flexible dataflow accelerator architecture for convolutional neural networks. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 553–564.
- [26] Zhichao Lu, Kalyanmoy Deb, Erik Goodman, Wolfgang Banzhaf, and Vishnu Naresh Boddeti. 2020. Nsganetv2: Evolutionary multi-objective surrogate-assisted neural architecture search. In *European Conference on Computer Vision*. Springer, 35–51.
- [27] Zhichao Lu, Ian Whalen, Vishnu Boddeti, Yashesh Dhebar, Kalyanmoy Deb, Erik Goodman, and Wolfgang Banzhaf. 2019. Nsga-net: neural architecture search using multi-objective genetic algorithm. In *Proceedings of the genetic and evolutionary computation conference*. 419–427.
- [28] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2018. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In *Proceedings of the European conference on computer vision (ECCV)*. 116–131.
- [29] Xiaohan Ma, Chang Si, Ying Wang, Cheng Liu, and Lei Zhang. 2021. NASA: accelerating neural network design with a NAS processor. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 790–803.
- [30] Jieru Mei, Yingwei Li, Xiaochen Lian, Xiaojie Jin, Linjie Yang, Alan Yuille, and Jianchao Yang. 2019. Atomnas: Fine-grained end-to-end neural architecture search. *arXiv preprint arXiv:1912.09640* (2019).
- [31] Naveen Muralimanohar, Rajeev Balasubramanian, and Norm Jouppi. 2007. Optimizing NUCA organizations and wiring alternatives for large caches with CACTI

- 6.0. In *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*. IEEE, 3–14.
- [32] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703* (2019).
- [33] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4510–4520.
- [34] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramanian, John Paul Strachan, Miao Hu, R Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 14–26.
- [35] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
- [36] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. 2017. Pipelayer: A pipelined reram-based accelerator for deep learning. In *2017 IEEE international symposium on high performance computer architecture (HPCA)*. IEEE, 541–552.
- [37] Dimitrios Stamoulis, Ruizhou Ding, Di Wang, Dimitrios Lymberopoulos, Bodhi Priyantha, Jie Liu, and Diana Marculescu. 2019. Single-path nas: Designing hardware-efficient convnets in less than 4 hours. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 481–497.
- [38] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1–9.
- [39] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. 2016. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2818–2826.
- [40] Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V Le. 2019. Mnasnet: Platform-aware neural architecture search for mobile. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2820–2828.
- [41] Mingxing Tan and Quoc Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*. PMLR, 6105–6114.
- [42] Mingxing Tan and Quoc Le. 2021. Efficientnetv2: Smaller models and faster training. In *International Conference on Machine Learning*. PMLR, 10096–10106.
- [43] Mingxing Tan and Quoc V Le. 2019. Mixconv: Mixed depthwise convolutional kernels. *arXiv preprint arXiv:1907.09595* (2019).
- [44] Qiyu Wan and Xin Fu. 2020. Fast-BCNN: Massive neuron skipping in Bayesian convolutional neural networks. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 229–240.
- [45] Qiyu Wan, Haojun Xia, Xingyao Zhang, Lening Wang, Shuaiwen Leon Song, and Xin Fu. 2021. Shift-BNN: Highly-efficient probabilistic Bayesian neural network training via memory-friendly pattern retrieving. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 885–897.
- [46] Chaoyue Wang, Chang Xu, Chaohui Wang, and Dacheng Tao. 2018. Perceptual adversarial networks for image-to-image transformation. *IEEE Transactions on Image Processing* 27, 8 (2018), 4066–4079.
- [47] Chaoyue Wang, Chang Xu, Xin Yao, and Dacheng Tao. 2019. Evolutionary generative adversarial networks. *IEEE Transactions on Evolutionary Computation* 23, 6 (2019), 921–934.
- [48] Hanrui Wang, Zhonghao Wu, Zhijian Liu, Han Cai, Ligeng Zhu, Chuang Gan, and Song Han. 2020. Hat: Hardware-aware transformers for efficient natural language processing. *arXiv preprint arXiv:2005.14187* (2020).
- [49] Peiqi Wang, Yu Ji, Chi Hong, Yongqiang Lyu, Dongsheng Wang, and Yuan Xie. 2018. Snrram: an efficient sparse neural network computation architecture based on resistive random-access memory. In *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [50] Bichen Wu, Xiaoliang Dai, Peizhao Zhang, Yanghan Wang, Fei Sun, Yiming Wu, Yuandong Tian, Peter Vajda, Yangqing Jia, and Kurt Keutzer. 2019. Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 10734–10742.
- [51] Xilinx. 2020. Xilinx Vivado. Retrieved June 8, 2021 from <https://www.xilinx.com/support/university/vivado.html>
- [52] Yuhui Xu, Lingxi Xie, Xiaopeng Zhang, Xin Chen, Guo-Jun Qi, Qi Tian, and Hongkai Xiong. 2019. Pc-darts: Partial channel connections for memory-efficient architecture search. *arXiv preprint arXiv:1907.05737* (2019).
- [53] Cheng-Xin Xue, Wei-Hao Chen, Je-Syu Liu, Jia-Fang Li, Wei-Yu Lin, Wei-En Lin, Jing-Hong Wang, Wei-Chen Wei, Ting-Wei Chang, Tung-Cheng Chang, et al. 2019. 24.1 A 1Mb multibit ReRAM computing-in-memory macro with 14.6 ns parallel MAC computing time for CNN based AI edge processors. In *2019 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 388–390.
- [54] Tzu-Hsien Yang, Hsiang-Yun Cheng, Chia-Lin Yang, I-Ching Tseng, Han-Wen Hu, Hung-Sheng Chang, and Hsiang-Pang Li. 2019. Sparse reram engine: Joint exploration of activation and weight sparsity in compressed neural networks. In *Proceedings of the 46th International Symposium on Computer Architecture*. 236–249.
- [55] Xiaoxuan Yang, Bonan Yan, Hai Li, and Yiran Chen. 2020. ReTransformer: ReRAM-based processing-in-memory architecture for transformer acceleration. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–9.
- [56] Shan You, Tao Huang, Mingmin Yang, Fei Wang, Chen Qian, and Changshui Zhang. 2020. Greedynas: Towards fast one-shot nas with greedy supernet. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 1999–2008.
- [57] Jiahui Yu, Pengchong Jin, Hanxiao Liu, Gabriel Bender, Pieter-Jan Kindermans, Mingxing Tan, Thomas Huang, Xiaodan Song, Ruoming Pang, and Quoc Le. 2020. Bignas: Scaling up neural architecture search with big single-stage models. In *European Conference on Computer Vision*. Springer, 702–717.
- [58] Chengming Zhang, Geng Yuan, Wei Niu, Jiannan Tian, Sian Jin, Donglin Zhuang, Zhe Jiang, Yanzhi Wang, Bin Ren, Shuaiwen Leon Song, et al. 2021. Click-train: Efficient and accurate end-to-end deep learning training via fine-grained architecture-preserving pruning. In *Proceedings of the ACM international conference on supercomputing*. 266–278.
- [59] Xingyao Zhang, Shuaiwen Leon Song, Chenhao Xie, Jing Wang, Weigong Zhang, and Xin Fu. 2020. Enabling highly efficient capsule networks processing through a PIM-based architecture design. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 542–555.
- [60] Zhen Zheng, Xuanda Yang, Pengzhan Zhao, Guoping Long, Kai Zhu, Feiwen Zhu, Wenyi Zhao, Xiaoyong Liu, Jun Yang, Jidong Zhai, et al. 2022. AStitch: enabling a new multi-dimensional optimization space for memory-intensive ML training and inference on modern SIMT architectures. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 359–373.
- [61] Zhao Zhong, Junjie Yan, Wei Wu, Jing Shao, and Cheng-Lin Liu. 2018. Practical block-wise neural network architecture generation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2423–2432.
- [62] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V Le. 2018. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 8697–8710.